What Can the Apple Neural Engine Do for Deep-Learning Training?

A Measurement Study with Energizer

Florian Grima

October 17, 2026

Abstract

The Apple Neural Engine (ANE) is the most power-efficient accelerator on Apple silicon, but it is reachable only through Core ML, which exposes no public backward pass. We measure what it can and cannot do for deep-learning training, on one Apple M4 Pro, with a reproducible harness and idle-subtracted per-rail power. Core ML places no fp32 operator on the ANE; in fp16 it places whole models on the ANE only above shape thresholds, and when it does the ANE draws about 2.7 W while the GPU path costs 1.3× the latency and 2.8× the energy on the configurations where both are possible. For training, the largest gain comes not from the ANE but from running the whole step (forward, backward, loss, optimiser) natively on the GPU through MLX: 4–8× faster than a NumPy CPU path. A hybrid ANE-forward/GPU-backward step never beats pure MLX on small models, because weight refresh requires a model recompilation of hundreds of milliseconds. The ANE does win when the weights do not change: fine-tuning a head on a frozen convolutional trunk runs 2.1–2.4× faster and uses about 10× less energy with the trunk on the ANE than on the GPU, at the same accuracy.

1 Introduction

Training on Apple silicon is usually discussed as "GPU or ANE". In practice the ANE cannot be programmed directly, the public stack gives inference only, and frameworks that advertise Apple acceleration (MLX, PyTorch MPS) run on the GPU. This paper asks three narrow, measurable questions: (i) when does Core ML place work on the ANE, and at what latency and power; (ii) how do Core ML, MLX and PyTorch-MPS compare against measured, not datasheet, peak throughput; (iii) what does an ANE-assisted training step cost compared to a pure GPU step. Contributions: an open measurement harness with clean-commit provenance; a placement/latency/power characterisation of 576 Core ML configurations; an MLX-native training path for Energizer with a per-optimisation ablation; and a measured hybrid-training overhead breakdown. All claims are tied to result files in `docs/claims.md`.

2 Background and Related Work

Apple's Neural Engine (ANE) is reachable only through Core ML [1]; MLX [3] and PyTorch's MPS backend [5] execute on the GPU through Metal. Apple documents ANE-oriented transformer design choices [2], and community reverse-engineering efforts describe ANE behaviour beyond the public documentation [4].

Table 1: Fraction of ops placed on the ANE by Core ML (compute units ALL).

workload	precision	n	median	min	max
conv	fp16	72	100%	0%	100%
conv	fp32	72	0%	0%	0%
matmul	fp16	24	100%	0%	100%
matmul	fp32	24	0%	0%	0%
transformer	fp16	48	100%	0%	100%
transformer	fp32	48	0%	0%	0%

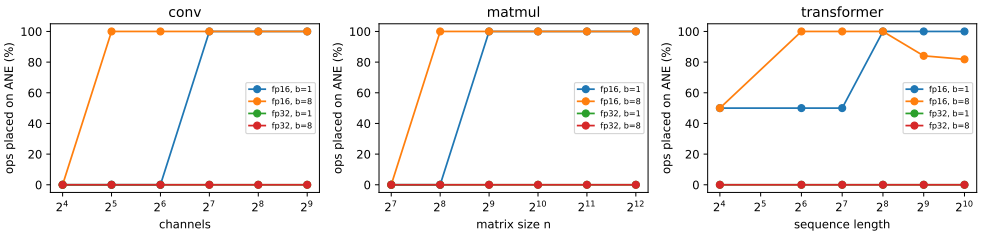


Figure 1: ANE placement fraction versus problem size.

3 Methodology

Hardware: Apple M4 Pro, macOS 26.6.2 (25G83), Python 3.12, MLX 0.30.6, coremltools 9.0, PyTorch 2.10. Every result record stores the environment and the git commit; runs on a dirty tree are flagged. Latency is the median of repeated warmed-up calls, with medians of repeats reported across repeats. Placement comes from `MLComputePlan` (planned, not observed, device). Power comes from `powermetrics` sampling at 100 ms per rail (CPU, GPU, ANE); energy is reported net of a measured idle baseline. Machine conditions matter: we observed 3–4× slowdowns on the GPU in two runs that shared the machine with other work, and repeated those runs on a quiet machine. Only runs that agree across repeats are reported. Full procedure: `docs/REPRODUCE.md`.

4 Characterisation of ANE Placement, Latency and Power

Core ML places no fp32 operator on the ANE (0 of 72 convolution, 24 matmul and 48 transformer configurations). In fp16 with `ALL` compute units, most configurations are fully placed on the ANE, and the exceptions follow shape thresholds (convolutions below roughly 128 channels at 64×64, sequences below roughly 128 at $d = 512$ stay off). Planned placement matches observed power: configurations planned on the ANE draw a median 2.7 W on the ANE rail, those not planned draw 0 W. On 31 batch-8 fp16 configurations that can run on both, the GPU path has a median 1.34× the latency and 2.82× the energy of the ANE path (Table 2). `ALL` sometimes splits a model between units and is then slower than a pure choice.

5 Baselines and Achieved-vs-Peak Efficiency

Table 3 gives the best measured square-matmul throughput per backend over $n \leq 4096$ (three repeats, within 3%). At large n MLX and PyTorch-MPS lead in fp16 (about 5.5–6.0 TFLOP/s)

Table 2: Median idle-subtracted power per rail and energy per inference (fp16).

workload	units	CPU mW	GPU mW	ANE mW	mJ/run
conv	ALL	4595	16	1717	5.860
conv	CPU_AND_GPU	4612	1196	1	13.906
conv	CPU_AND_NE	4467	14	1545	5.970
conv	CPU_ONLY	5681	14	0	7.323
matmul	ALL	4484	7	1355	5.622
matmul	CPU_AND_GPU	3536	2343	2	18.745
matmul	CPU_AND_NE	4427	7	1773	5.668
matmul	CPU_ONLY	4690	7	0	5.632
transformer	ALL	1894	33	2652	3.732
transformer	CPU_AND_GPU	1754	12581	3	12.234
transformer	CPU_AND_NE	1571	24	2808	2.495
transformer	CPU_ONLY	6142	30	0	5.538

Table 3: Best measured square-matmul throughput (denominator for achieved-vs-peak).

backend	precision	peak TFLOP/s
coreml_cpu_gpu	fp16	4.00
coreml_cpu_gpu	fp32	4.36
coreml_cpu_ne	fp16	5.17
coreml_cpu_ne	fp32	3.20
mlx	fp16	6.01
mlx	fp32	5.36
torch_mps	fp16	5.52
torch_mps	fp32	4.84

and the Core ML ANE path reaches about 5.2 TFLOP/s; Core ML on the GPU is lowest (about 4 TFLOP/s). At small sizes, Core ML dispatch dominates (Table 4) and MLX/MPS are faster in absolute latency, except on the transformer workload where Core ML with the ANE is fastest.

6 Training Paths

MLX-native step. Moving forward, backward, loss and optimiser to MLX cuts the Adam step from 8.9 to 1.9 ms (MLP, width 1024), 40 to 7.5 ms (width 2048), 35.6 to 4.6 ms (CNN) and 29.7 to 7.2 ms (transformer) relative to the NumPy CPU path. The ablation shows the optimiser is the largest single factor: keeping only the optimiser on the host raises a width-2048 step from 7.4 to 23.7 ms; keeping only the loss on the host costs 3%.

Hybrid. Core ML forward takes 0.2–0.7 ms and transfers 0.01–0.03 ms, but the backward and optimiser pass on the GPU takes 0.4–0.8 ms and refreshing the Core ML model after a weight update takes 260–320 ms (Table 5). Hybrid never beats pure MLX at widths 256–1024.

Convergence. On MNIST (MLP, 3 epochs, same seed) CPU, MLX and hybrid fp16 with a weight refresh every step reach 93.00%, 93.02% and 92.98% (Table 6); the hybrid path takes about

Table 4: Median inference latency per framework.

workload	precision	framework	n	median ms
conv	fp16	coreml_all	36	0.820
conv	fp16	coreml_cpu_gpu	36	1.202
conv	fp16	coreml_cpu_ne	36	0.832
conv	fp16	mlx	36	0.517
conv	fp16	torch_mps	36	0.407
conv	fp32	coreml_all	36	0.911
conv	fp32	coreml_cpu_gpu	36	0.900
conv	fp32	mlx	36	0.635
conv	fp32	torch_cpu	36	1.430
conv	fp32	torch_mps	36	0.438
matmul	fp16	coreml_all	12	1.239
matmul	fp16	coreml_cpu_gpu	12	1.561
matmul	fp16	coreml_cpu_ne	12	1.241
matmul	fp16	mlx	12	0.509
matmul	fp16	torch_mps	12	0.548
matmul	fp32	coreml_all	12	0.900
matmul	fp32	coreml_cpu_gpu	12	0.962
matmul	fp32	mlx	12	0.551
matmul	fp32	torch_cpu	12	0.718
matmul	fp32	torch_mps	12	0.585
transformer	fp16	coreml_all	24	0.685
transformer	fp16	coreml_cpu_gpu	24	0.793
transformer	fp16	coreml_cpu_ne	24	0.531
transformer	fp16	mlx	24	0.626
transformer	fp16	torch_mps	24	0.835
transformer	fp32	coreml_all	24	0.778
transformer	fp32	coreml_cpu_gpu	24	0.776
transformer	fp32	mlx	24	0.771
transformer	fp32	torch_cpu	24	1.780
transformer	fp32	torch_mps	24	0.923

135 s versus 0.2–0.4 s, and most of its energy is on the CPU rail. Refreshing every 5 or 20 steps diverged to NaN in our setting; this is a negative result of the stale-weight approach, not of the ANE. In the energy breakdown of the refresh-every-step run, the CPU rail accounts for 690 J and the ANE rail for 63 J of 737 J. A first run of the same experiment attributed 0.5 J to the ANE; we have not explained that difference, so we make no claim about the ANE share of hybrid energy.

Frozen trunk. The hybrid fails because the ANE model must be rebuilt after every update. If the trunk never changes, there is nothing to rebuild. We pre-train a trunk on MNIST digits 0–4, freeze it, and train a new linear head on digits 5–9 (Adam, 5 epochs, batch 256 unless noted). The trunk is compiled to Core ML once. We compare the trunk forward on the MLX GPU (fp32 and fp16) with Core ML on the ANE (placement 100% ANE for every trunk), run online every step, pipelined (the ANE predicts batch $i{+}1$ while the GPU trains the head on batch i), or once into a feature cache (Tables 7 and 8, Figure 7). Every head reaches the same test accuracy whatever runs

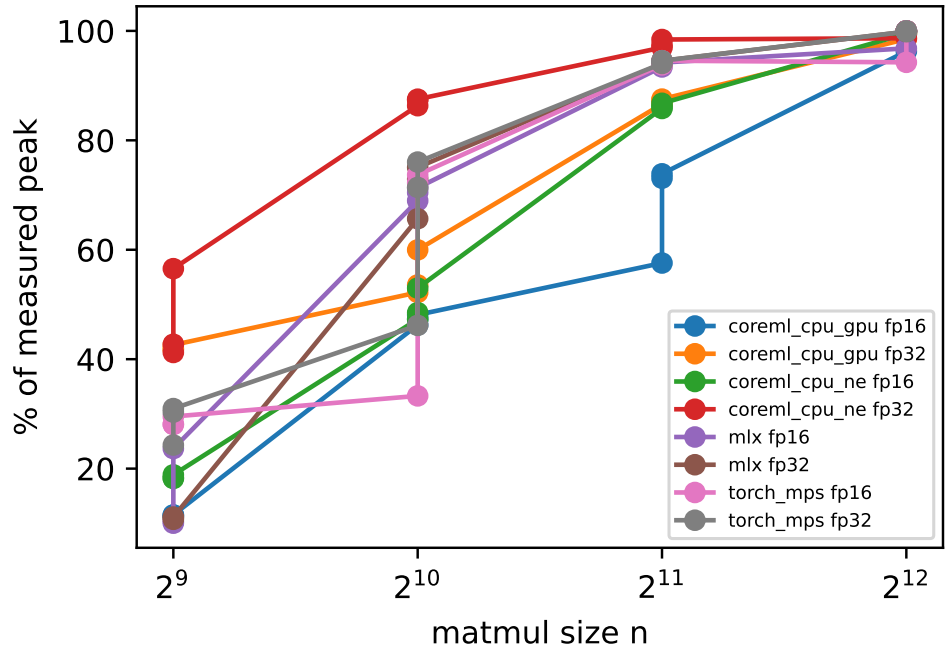


Figure 2: Achieved fraction of measured peak.

Table 5: Hybrid step overhead breakdown (ms).

width	units	fwd	xfer	bwd+opt	pure MLX	sync (ms)
256	ALL	0.36	0.01	0.64	0.85	270
256	CPU_AND_NE	0.21	0.01	0.56	0.35	259
512	ALL	0.30	0.01	0.40	0.49	299
512	CPU_AND_NE	0.28	0.01	0.58	0.47	268
1024	ALL	0.69	0.02	0.74	1.07	284
1024	CPU_AND_NE	0.49	0.03	0.81	1.12	320

the trunk (within 0.05 points, fp16 trunk features within 0.1% of fp32 for the Core ML models and at most 0.8% for MLX fp16). The gain grows with trunk compute and is largest for convolutions: on the widest CNN (64–256 channels, batch 256) the ANE run takes 3.1 s against 6.6 s for MLX fp16 (2.1×) and 8.8 s for fp32 (2.9×), and 12 J against 136 J (11×) and 214 J (17×). The ANE rail carries almost all of that energy and the GPU is idle. With cached features the ANE extracts them 2.1× faster at 9× lower energy; after extraction, all paths are identical. Pipelining the ANE with the GPU head step adds 1.2–1.6× over the unpipelined ANE run, most when both stages have comparable cost (MLPs, small CNN) and least when the ANE stage dominates (widest CNN, 1.2×). For the small MLP (width 512) the ANE gives 1.1× in time and 2× in energy, so there is a size below which the ANE is not worth using for speed. The MLP heads reach only 50–77% accuracy because the brief digits-0–4 pre-training transfers poorly through deep MLP features; their timing and energy are valid, their accuracy is not a claim. Two independent runs (timing only, and with power) agree to within 1–20% on every configuration, and the same-commit caveat of Section 8 applies.

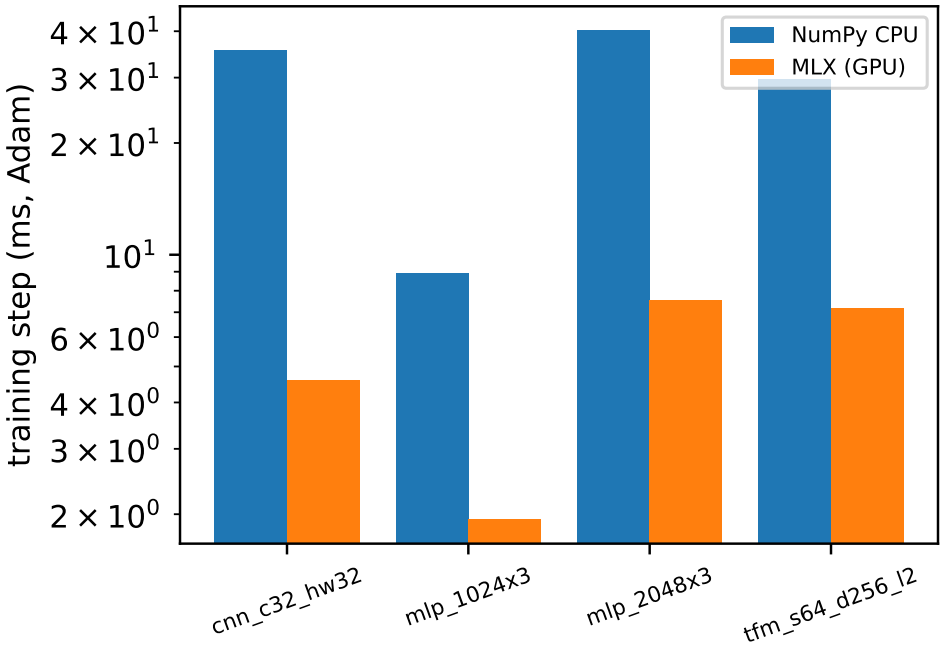


Figure 3: Training-step latency, CPU versus MLX.

Table 6: Same seed and data: final loss, accuracy, wall time and idle-subtracted energy per training run.

path	final loss	test acc	time s	energy J
cpu	0.1643	93.00%	0.4	2.3
hybrid_fp16_k1	0.1645	92.98%	135.1	737.3
mlx	0.1644	93.02%	0.2	2.1

7 Conclusion

We set out to learn what the ANE can do for deep-learning training. Starting expectations were: (i) that frameworks advertised as Apple-accelerated use the ANE; (ii) that the ANE would be the fastest place to run a model whenever Core ML allows it; (iii) that placing the forward pass on the ANE and the backward pass on the GPU would lower the cost of a training step; and (iv) that this hybrid would reach the same accuracy as fp32 training.

Answer. On this hardware and software stack the ANE is an efficient engine for *fixed-weight, fp16 inference of sufficiently large operators*, not a general training accelerator. It cannot execute a backward pass, and every weight update forces a model rebuild that dominates the step. It does help for the part of training that does not change: with a frozen convolutional trunk it doubled the speed and cut energy by an order of magnitude. The 4–8× training speed-up we obtained came from keeping the entire step, including the optimiser, native on the GPU with MLX, not from the ANE.

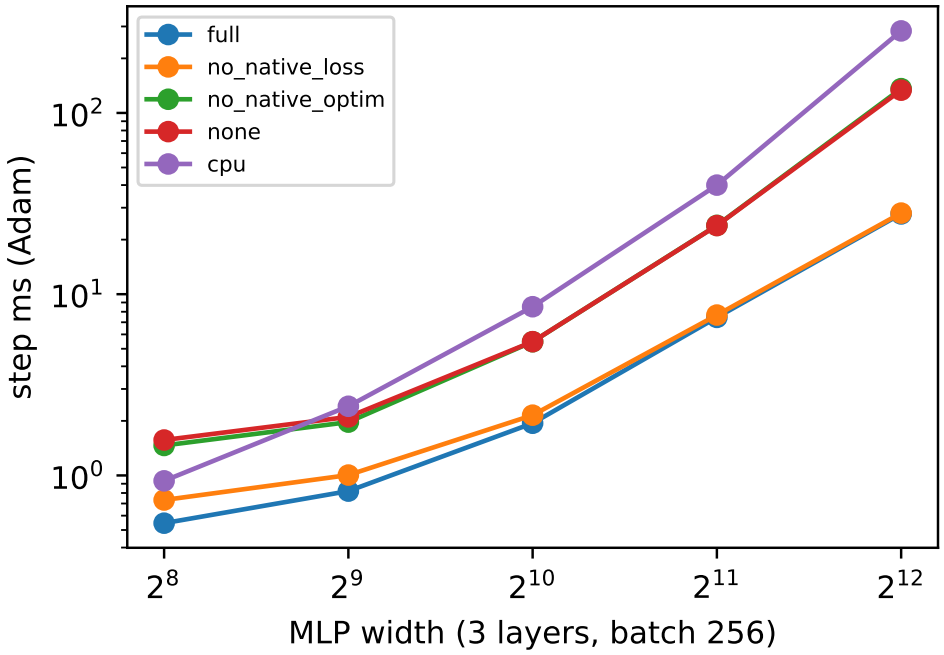


Figure 4: Ablation: step time as each MLX-native component is moved to the host.

What this does not show. Larger models, other chips and later OS releases may move the placement thresholds and the hybrid's break-even point. The frozen-trunk test used one small transfer task (MNIST digits), trunks up to about 31M parameters and a hand-written MLX forward that we did not compile with `mx.compile`, so a tuned GPU baseline could narrow the gap. Pre-trained models of realistic size (for example vision transformers) are untested. Whether private interfaces would change the answer is left open; we used none.

8 Limitations

Single chip and OS build, one machine; Core ML placement is a planner decision that may differ at run time; no public ANE backward pass exists, so the hybrid path pays a full recompilation per weight refresh; results are for small models and inference-style forward passes; no private APIs were used. Large batch-8 matmuls ($n \geq 1024$) varied by 1.4–5× between two full sweeps on the same commit, including pure-CPU rows, so we quote medians over many configurations rather than single large-matmul rows. The environment record of the power runs reports battery power although the machine was reportedly plugged in; we report it as recorded. Later runs record AC power and charge level in the environment. The frozen-trunk runs were made before their scripts were committed, so their result files carry a dirty-tree flag.

9 Reproducibility

All numbers come from raw JSON-lines files in `results/`, written by the scripts in `experiments/` (one script per experiment, `make reproduce` runs them all); tables and figures are generated from those files and never edited by hand. Each file starts with an environment record: chip, macOS

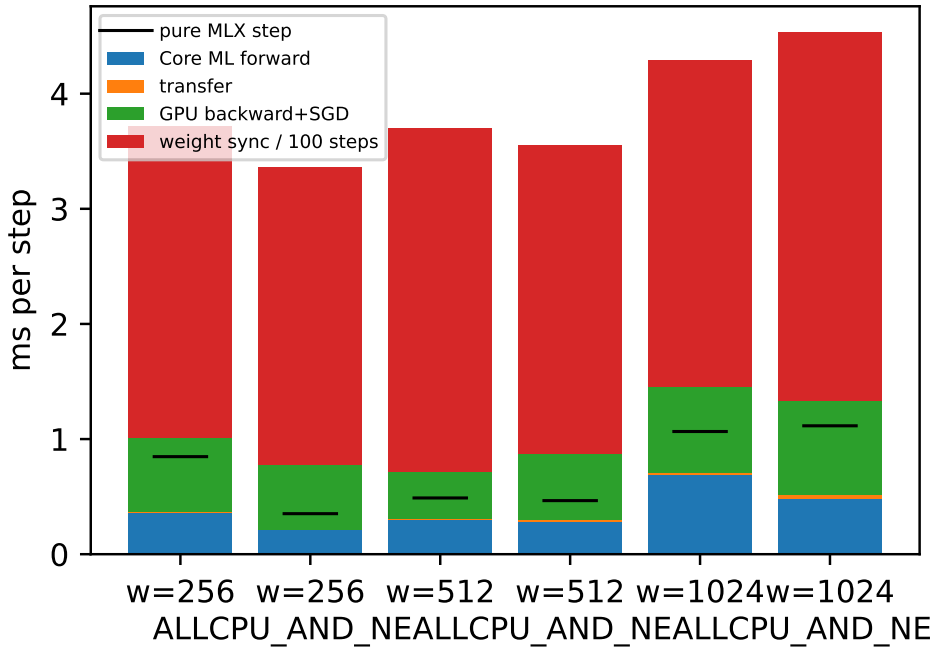


Figure 5: Hybrid step breakdown against a pure MLX step.

build, Python, NumPy, MLX, PyTorch and coremltools versions, git commit and dirty flag, AC or battery power, charge level, Low Power Mode and thermal limits. The reference machine is an Apple M4 Pro (macOS 26.6.2, build 25G83, Python 3.12.12, MLX 0.30.6, coremltools 9.0, PyTorch 2.10.0); the pinned package list is in `environment/requirements.lock`. Seeds are fixed (data order, initialisation, pre-training), MNIST is downloaded through torchvision, and timing reports medians over repeated runs after untimed warm-up. Power and energy need `powermetrics` (root), subtract a measured idle baseline, and are reported per rail. Two runs were discarded because the machine was busy (three to four times slower GPU paths); a sanity gate (MLX fp16 4096^3 matmul near 23 ms) now identifies such runs. Energy and latency cannot be bit-identical across runs; we therefore report spread where we have repeats and quote medians over many configurations elsewhere. Code, raw results and the build recipe are in the repository.

References

[1] Apple Inc. Core ML and coremltools documentation. `https://apple.github.io/coremltools/`. coremltools 9.0 used here. VERIFY: access date.

[2] Apple Machine Learning Research. Deploying transformers on the Apple Neural Engine. `https://machinelearning.apple.com/research/neural-engine-transformers`, 2022. VERIFY: access date.

[3] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: An array framework for Apple silicon. `https://github.com/ml-explore/mlx`, 2023. Version 0.30.6 used here. VERIFY: author list.

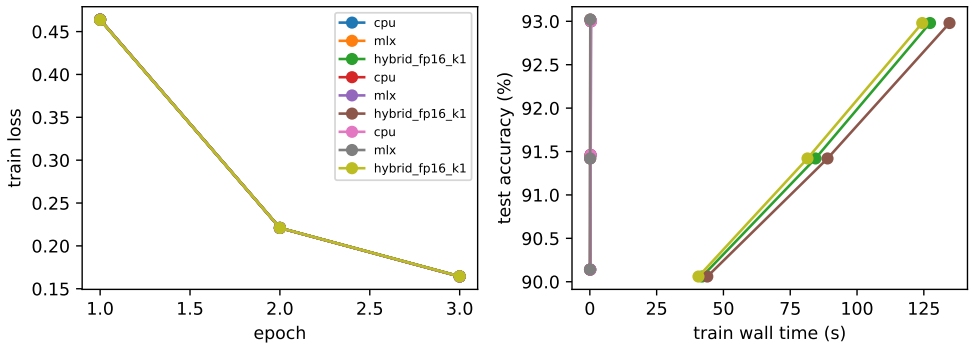


Figure 6: Convergence of the three paths on MNIST.

Table 7: Frozen-trunk fine-tuning (5 epochs, online): wall time and idle-subtracted energy per run. Trunk on the MLX GPU (fp32, fp16) vs Core ML on the ANE (alone, and pipelined with the GPU head step). Speed and energy columns are MLX fp16 divided by ANE; accuracy is the head's test accuracy.

trunk	fp32 s	fp16 s	ANE s	ANE+pipe s	speed	fp16 J	ANE J	energy	acc
mlp-w512-d4-b256	0.41	0.40	0.37	0.23	1.1×	3.6	1.6	2×	77.2%
mlp-w1024-d6-b256	0.83	0.79	0.50	0.32	1.6×	11.0	2.4	5×	64.6%
mlp-w2048-d8-b256	2.48	2.22	1.19	0.78	1.9×	50.5	4.3	12×	55.9%
mlp-w2048-d8-b64	3.92	3.63	2.70	1.83	1.3×	55.4	7.1	8×	50.0%
cnn-w16-d6-b256	1.52	1.38	0.57	0.38	2.4×	25.7	2.6	10×	89.5%
cnn-w32-d6-b256	3.16	2.52	1.12	0.85	2.3×	53.2	5.2	10×	94.3%
cnn-w64-d6-b256	8.84	6.59	3.09	2.65	2.1×	135.8	12.3	11×	93.9%

[4] Matthijs Hollemans. Everything we actually know about the Apple Neural Engine. `https://github.com/hollance/neural-engine`. Community repository. VERIFY: year, whether still maintained.

[5] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019.

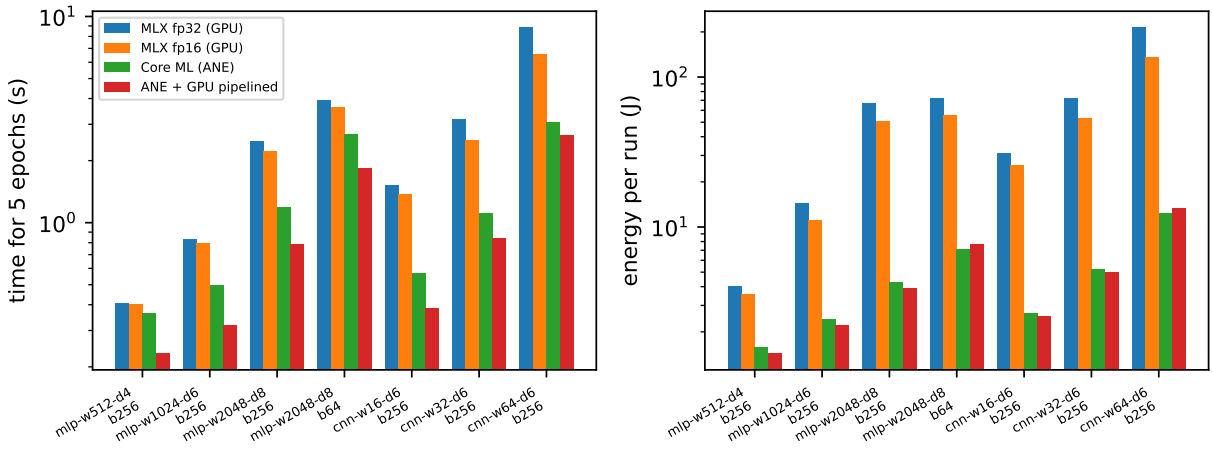


Figure 7: Frozen-trunk fine-tuning: time and energy per run for four trunk placements.

Table 8: Frozen trunk with cached features: features extracted once, then 5 epochs on the cache.

trunk	fp16 s	ANE s	speed	fp16 J	ANE J
mlp-w512-d4-b256	0.20	0.20	1.0×	1.4	1.0
mlp-w1024-d6-b256	0.29	0.22	1.3×	3.1	1.2
mlp-w2048-d8-b256	0.55	0.30	1.8×	12.3	1.9
mlp-w2048-d8-b64	1.17	0.94	1.2×	15.2	4.4
cnn-w16-d6-b256	0.39	0.22	1.8×	6.3	1.1
cnn-w32-d6-b256	0.63	0.32	2.0×	12.5	1.7
cnn-w64-d6-b256	1.46	0.69	2.1×	30.7	3.5

Table 9: Expectations against measurements on one Apple M4 Pro.

expected	measured
MLX and MPS use the ANE	They run on the GPU; the ANE is reachable only through Core ML.
ANE usable for any model	No fp32 operator is planned on the ANE; in fp16 only above shape thresholds.
ANE is the fastest unit	Peak matmul 5.2 TFLOP/s against 6.0 (MLX); fastest only on the transformer workload.
ANE is more efficient	Confirmed: median 2.7 W on the ANE rail; the GPU path costs 1.34× latency and 2.82× energy on 31 configurations.
Hybrid step is cheaper	Refreshing Core ML weights costs 260–320 ms; never beats pure MLX at widths 256–1024.
Hybrid matches fp32 accuracy	Only with a refresh every step (92.98% against 93.00%) at about 300× the energy; refreshing every 5 or 20 steps diverged.
ANE helps when weights are frozen	Confirmed: frozen CNN trunk 2.1–2.4× faster and 10–11× less energy than MLX fp16, same accuracy; 1.1× on the smallest MLP.